

# RuFLEX: final article text

## Название

RuFLEX: гибридная платформа глубокого нечеткого обучения для геоданных с интерпретируемым анализом правил, факторов и скрытых концептов

## Аннотация

В работе представлена платформа RuFLEX для гибридного глубокого нечеткого обучения, ориентированная на обработку геоданных в задачах, где высокая прогностическая точность должна сочетаться с интерпретируемостью, экспертной верифицируемостью и прозрачностью логики вывода. В отличие от традиционных непрозрачных подходов предложенная архитектура объединяет дифференцируемые нечеткие слои, модули глубокого нечеткого формирования признаков, расширяемый пространственный контур глубоких нечетких сверток и механизмы принятия решений на основе правил в едином дифференцируемом вычислительном контуре, реализованном в экосистеме PyTorch. Модельный контур поддерживает совместное обучение параметров функций принадлежности, базы правил и глубоких представлений признаков при сохранении семантически интерпретируемого вывода на уровне факторов и правил. Платформа включает программное ядро в виде программного интерфейса Python, веб-интерфейс визуального проектирования моделей и панель интерпретации, обеспечивающую анализ функций принадлежности, структуры правил, скрытых нечетких концептов и вклада факторов в итоговое решение. Перспективным направлением развития является расширение контура объяснимости в сторону логической прослеживаемости правил и контрфактуального анализа.

## 1. Введение

В задачах геоаналитики, пространственной классификации, прогнозирования и интеллектуальной поддержки принятия решений высокая точность предсказания не может рассматриваться отдельно от интерпретируемости модели. Для прикладных постановок существенны итоговые значения метрик, прозрачность логики вывода, возможность экспертной проверки базы правил и понимание того, какие факторы определили ответ на конкретном объекте. По этой причине непрозрачные архитектуры, хотя и способны давать высокое качество прогноза, не снимают задачу полностью.

Нечеткие и нейро-нечеткие методы, напротив, сохраняют связь между обучаемым представлением признаков и семантически понятным выводом, поэтому продолжают занимать важное место в исследованиях такого типа [1].

Современная постановка задачи требует уже не отдельной модели, а целостной программной среды. Если исследователь хочет сравнить компактную интерпретируемую конфигурацию и более глубокий контур формирования признаков, ему недостаточно иметь только вычислительное ядро. Нужны средства задания переменных, термов, функций принадлежности и правил, средства обучения, визуального проектирования, воспроизводимой постановки вычислительного опыта и содержательной интерпретации результата. Именно эта логика определяет платформенный характер RuFLEX. В настоящей работе она рассматривается не как единичный вариант нейро-нечеткой сети, а как расширяемая среда для гибридного глубокого нечеткого обучения.

Цель статьи состоит в описании архитектуры платформы RuFLEX, формализации ее модельного контура и прикладной демонстрации работы на задачах обработки геоданных. Основной акцент делается на разделении вычислительного и платформенного контуров, на модульной организации платформы, на совместном обучении параметров функций принадлежности, базы правил и внутренних представлений, а также на встроенных средствах интерпретации, которые позволяют анализировать поведение модели на уровне признаков, правил и скрытых нечетких концептов.

## **2. Архитектура и модульный состав платформы**

RuFLEX реализована как платформенный слой поверх вычислительного ядра *ruanfis*, происходящего из репозитория *deer-neuro-fuzzy* [7]. Такое построение выбрано сознательно. Вычислительное ядро отвечает за дифференцируемые нечеткие преобразования, вычисление активаций правил, локальные правилые блоки, базовые процедуры обучения и исполнение модельного контура. Платформенный слой развивает собственную объектную модель проекта, сериализацию, интерфейсную оболочку, средства визуализации, управление вычислительными опытами и контур интерпретации. Благодаря этому сохраняется преемственность глубокого нечеткого ядра, а сама платформа развивается как самостоятельная исследовательская и прикладная среда.

Архитектура платформы строится как совокупность двух взаимосвязанных контуров. Первый контур является вычислительным. В нем располагаются

фаззификация, вычисление активаций правил, построение скрытых концептов и формирование решающего слоя. Второй контур является платформенным. Он охватывает описание данных, объект проекта, средства настройки моделей, журналирование обучения, визуальное сопровождение экспериментов, анализ результатов и экспорт материалов. Такое разделение принципиально важно, поскольку RuFLEX заявляется именно как платформа, а не как единичная реализация одной модели.

На семантическом уровне проекта задаются входные и выходные переменные, диапазоны их значений, роли признаков, режимы нормализации и принадлежность переменной к входному, скрытому или выходному уровню. Здесь же формируются термы и функции принадлежности. Для каждой переменной могут быть заданы гауссовы, колоколообразные, треугольные и трапециевидные функции принадлежности. Для дифференцируемых режимов основную роль играют гладкие семейства, пригодные для совместной оптимизации. Тем самым платформа связывает предметное описание данных с обучаемым нечетким представлением.

Следующий уровень образует правилковая подсистема. В ней задаются антецеденты, типы агрегации, веса правил, параметры заключений, локальные базы правил и ограниченные блоки, препятствующие взрывному росту числа правил. Локальная блочная организация является принципиальной особенностью платформы, поскольку именно она делает возможным переход от плоского интерпретируемого режима к более глубокому построению признаков без разрушения прозрачности вывода. На этой основе формируется модельный слой, который включает плоскую интерпретируемую конфигурацию, режим глубокого нечеткого формирования признаков и расширяемый пространственный каркас.

Подсистема обучения объединяет начальную инициализацию, поэтапное обучение, циклы уточнения и совместную оптимизацию параметров функций принадлежности, правил и внутренних представлений. Она дополняется журналированием, сохранением лучших состояний, историей обучения и воспроизводимым запуском вычислительных опытов.

Подсистема интерпретации обеспечивает визуализацию функций принадлежности, анализ структуры правил, активаций и нормированных весов, анализ вклада правил в скрытые концепты и вклада скрытых концептов в итоговый ответ. На уровне пользовательского контура эти средства переходят в программный интерфейс Python, веб-интерфейс

визуального проектирования, панель интерпретации и экспорт артефактов статьи. Схема 1 фиксирует разделение вычислительного ядра и платформенных сервисов, схема 2 показывает общий модельный контур, а схема 3 описывает путь от выбранного объекта к аналитическим материалам.

### 3. Математическая структура модельного контура

Пусть входной объект описывается вектором признаков  $x = (x_1, \dots, x_d)$ , а нормализованное представление входа задается вектором  $z^{(0)}(x) = (z_1^{(0)}, \dots, z_d^{(0)})$ , где  $z_i^{(0)} = \varphi_i(x_i)$  и  $\varphi_i$  обозначает отображение нормализации. Для  $i$ -й переменной и  $j$ -го терма задается функция принадлежности  $\mu_{ij}(z_i^{(0)}; \theta_{ij})$ , где  $\theta_{ij}$  обозначает набор настраиваемых параметров. Для гауссовой функции принадлежности используется выражение

$$\mu_{ij}(z_i^{(0)}) = \exp(- (z_i^{(0)} - c_{ij})^2 / (2\sigma_{ij}^2)),$$

а для колоколообразной функции принадлежности

$$\mu_{ij}(z_i^{(0)}) = 1 / (1 + |(z_i^{(0)} - c_{ij}) / a_{ij}|^{2b_{ij}}).$$

После фаззификации формируется набор локальных степеней принадлежности, на основе которого вычисляются активации правил. Для  $r$ -го правила на уровне  $l$  его ненормированная активация вычисляется как

$$w_r^{(l)}(x) = \prod_{m=1}^{k_r} \mu_{jm}(z_m^{(l-1)}(x); \theta_{jm}^{(l)}),$$

где  $z^{(l-1)}(x)$  обозначает представление предыдущего уровня, а  $k_r$  есть длина правила. Нормированный вес правила определяется выражением

$$\bar{w}_r^{(l)}(x) = w_r^{(l)}(x) / (\sum_s w_s^{(l)}(x) + \varepsilon),$$

где  $\varepsilon > 0$  предотвращает вырождение знаменателя. В плоской интерпретируемой конфигурации именно эти нормированные веса непосредственно участвуют в вычислении отклика решающего слоя.

В более общем случае, который соответствует глубокому нечеткому формированию признаков, входное пространство разбивается на локальные блоки  $B_q^{(l)}$ , каждый из которых формирует новый скрытый нечеткий концепт. Для  $q$ -го концепта на уровне  $l$  используется выражение

$$h_q^{(l)}(x) = \sum_{r \in B_q^{(l)}} \bar{w}_r^{(l)}(x) \cdot g_r^{(l)}(z^{(l-1)}(x)),$$

где  $B_q^{(l)}$  есть множество правил блока, формирующего  $q$ -й концепт, а  $g_r^{(l)}$  задает параметризованное заключение правила. Тогда вектор  $h^{(l)}(x)$  образует новое пространство признаков, которое подается на следующий уровень модели. Финальный решающий слой вычисляет ответ

$$\hat{y}(x) = \sum_{r \in B^L} \bar{w}_r^L(x) \cdot g_r^L(h^{L-1}(x)).$$

Такое представление позволяет сохранить интерпретируемость на уровне правил и одновременно наращивать выразительность модели за счет последовательного построения скрытых нечетких концептов. Скрытый слой при этом трактуется не как непрозрачный внутренний вектор, а как новое концептуальное пространство, полученное из ограниченного множества локальных правил.

Совместное обучение параметров платформы задается функционалом

$$L = L_{\text{задачи}} + \lambda_1 L_{\text{разреженности}} + \lambda_2 L_{\text{перекрытия}} + \lambda_3 L_{\text{покрытия}} + \lambda_4 L_{\text{концептов}},$$

где  $L_{\text{задачи}}$  соответствует основной метрике качества, а остальные слагаемые регулируют компактность базы правил, структуру термов и различимость скрытых концептов. В простейшем виде эти слагаемые могут быть заданы как

$$L_{\text{разреженности}} = \sum_{l,r} |\alpha_r^l|,$$

$$L_{\text{перекрытия}} = \sum_{i,j < k} \int \mu_{ij}(t) \mu_{ik}(t) dt,$$

$$L_{\text{концептов}} = \sum_l \sum_{(q \neq p)} < h_q^l, h_p^l >^2.$$

Здесь  $\alpha_r^l$  обозначает вес  $r$ -го правила, а выражение для  $L_{\text{концептов}}$  препятствует вырождению скрытых концептов в близкие по смыслу представления. Благодаря этому обучение ориентировано на качество прогноза и на структурную интерпретируемость модели.

Для пространственного контура вводится тензор признаков  $X \in R^{(H \times W \times C)}$ . В этом случае нечеткий сверточный отклик  $m$ -го ядра на позиции  $(u, v)$  может быть записан в виде

$$u_m(u, v) = \sum_{(a,b,c) \in \Omega} \kappa_{(mabc)} \cdot \mu_{(mabc)}(X(u+a, v+b, c)),$$

где  $\Omega$  задает локальное окно,  $\kappa_{(mabc)}$  являются обучаемыми весами, а  $\mu_{(mabc)}$  описывают локальные функции принадлежности. В первой версии платформы этот модуль подготовлен как расширяемый каркас, согласованный с основной архитектурой и предназначенный для дальнейшего развития пространственного контура.

Интерпретация результата строится через вклад правил, концептов и факторов. Для  $r$ -го правила в финальном слое локальный вклад может быть оценен как

$$C_r(x) = \bar{w}_r^L(x) \cdot g_r^L(h^{L-1}(x)),$$

а вклад  $q$ -го скрытого концепта в итоговый ответ задается выражением

$$C_q^{(l)}(x) = \sum_{r \in B_q^{(l)}} C_r^{(l)}(x).$$

После этого вклад исходных признаков восстанавливается через агрегирование по тем блокам и правилам, в которых эти признаки участвовали. В результате интерпретация опирается на внутреннюю структуру самого модельного контура, а не на внешний постфактум-анализ.

## 4. Экспериментальная постановка

Для прикладной проверки платформы использованы две задачи обработки геоданных на основе набора `sklearn.fetch_california_housing` [3, 4]. Первая задача, `california_housing_regression`, является задачей регрессии по переменной `MedHouseVal` при сохранении пространственных признаков `Latitude` и `Longitude`. Вторая задача, `california_value_binary_geo`, строится на том же источнике данных в постановке бинарной классификации, где целевая переменная `HighValue` определяется порогом 1.797 по исходной величине `MedHouseVal`. В обеих задачах использовались подвыборки объема 4096 наблюдений. Для регрессионного набора применялось случайное подвыборочное извлечение при `random_state = 42`, а для задачи классификации использовалась сбалансированная подвыборка при том же значении параметра.

Во всех вычислительных опытах использовалось разбиение данных на обучающую, проверочную и тестовую части с долями 0.6, 0.2 и 0.2 соответственно. Сравнивались четыре конфигурации платформы: плоская базовая конфигурация, плоская интерпретируемая конфигурация, демонстрационный глубокий режим и расширенный глубокий режим. Плоские варианты обучались в режиме начальной инициализации с последующим дообучением, а глубокие конфигурации использовали поэтапное предварительное обучение с последующим дообучением. Для регрессионной задачи анализировались метрики RMSE, MAE и  $R^2$ , для классификационной задачи использовались `accuracy`, `precision`, `recall` и `F1`. Все эксперименты выполнялись в воспроизводимом исследовательском контуре, который автоматически сохраняет метрики, таблицы результатов, графики, панели интерпретации и текстовые отчеты.

## 5. Результаты и интерпретация

Результаты на задаче `california_housing_regression` являются основным количественным аргументом статьи. Лучшей оказалась конфигурация

«расширенный глубокий режим», для которой получены  $\text{test\_rmse} = 0.795757$ ,  $\text{test\_mae} = 0.593770$  и  $\text{test\_r2} = 0.507159$ . Для конфигурации «плоская базовая конфигурация» соответствующие значения составили  $\text{test\_rmse} = 0.890950$ ,  $\text{test\_mae} = 0.678868$  и  $\text{test\_r2} = 0.382192$ . Уже демонстрационный глубокий режим показал  $\text{test\_rmse} = 0.816314$  и  $\text{test\_r2} = 0.481367$ , то есть переход от плоской конфигурации к глубокому нечеткому построению признаков дает устойчивое улучшение качества, а расширение архитектуры усиливает этот эффект. Именно поэтому регрессионный сценарий выступает главным экспериментом статьи.

Результаты на задаче `california_value_binary_geo` решают другую методологическую задачу. Здесь различия между конфигурациями оказываются существенно более сдержанными, а лучшей оказывается «плоская интерпретируемая конфигурация», для которой получены  $\text{test\_accuracy} = 0.746341$ ,  $\text{test\_precision} = 0.743073$ ,  $\text{test\_recall} = 0.735661$  и  $\text{test\_f1} = 0.739348$ . Для сравнения, «расширенный глубокий режим» дал  $\text{test\_accuracy} = 0.743902$  и  $\text{test\_f1} = 0.736181$ , а «демонстрационный глубокий режим» показал  $\text{test\_accuracy} = 0.739024$  и  $\text{test\_f1} = 0.738386$ . Этот результат важен именно для платформенного тезиса статьи. RuFLEX не навязывает одну архитектуру как заведомо лучшую во всех постановках. В регрессии преимущество получает более глубокий режим, тогда как в классификации лучший баланс качества и прозрачности дает компактная интерпретируемая конфигурация.

Содержательная ценность платформы определяется не только итоговыми метриками. Для лучших конфигураций автоматически формируются панели интерпретации, графики функций принадлежности, история обучения, представления наиболее активных правил, скрытых нечетких концептов и траекторий их формирования. Поэтому статья получает не просто сводку численных результатов, а структурный материал, позволяющий проследить, каким образом итоговый ответ модели связан с активированными правилами, степенями принадлежности и промежуточными концептами. В рецензируемой публикации это особенно важно, поскольку сильный количественный результат подкрепляется интерпретируемым устройством модели и воспроизводимой процедурой вычислительного опыта.

## 6. Заключение

В статье представлена платформа RuFLEX для гибридного глубокого нечеткого обучения, ориентированная на обработку геоданных и объединяющая дифференцируемые нечеткие слои, модули глубокого

нечеткого формирования признаков, расширяемый пространственный контур глубоких нечетких сверток и механизмы принятия решений на основе правил. Показано, что платформа позволяет совместно обучать параметры функций принадлежности, базы правил и внутренних представлений признаков, сохраняя интерпретируемый вывод на уровне факторов, правил и скрытых концептов.

Экспериментальная проверка на задачах регрессии и бинарной классификации подтверждает, что RuFLEX следует рассматривать именно как платформу выбора и сравнения модельных режимов. На регрессионной задаче глубокий режим построения нечетких признаков демонстрирует убедимое преимущество над плоской базовой конфигурацией. На классификационной задаче лучший результат достигается интерпретируемой плоской конфигурацией. Тем самым платформа показывает свою сильную сторону именно там, где требуется сопоставление архитектур при сохранении прозрачности вывода и воспроизводимости вычислительного опыта. Перспективы дальнейшего развития связаны с усилением пространственного контура, расширением логической прослеживаемости правил и развитием контрфактуальной интерпретации без изменения базовой архитектурной схемы.

## 9. References

1. Jang J.-S. R. ANFIS: Adaptive-Network-Based Fuzzy Inference System // IEEE Transactions on Systems, Man, and Cybernetics. 1993. Vol. 23, No. 3. P. 665-685.
2. Paszke A., Gross S., Massa F. et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library // Advances in Neural Information Processing Systems. 2019. Vol. 32.
3. Pedregosa F., Varoquaux G., Gramfort A. et al. Scikit-learn: Machine Learning in Python // Journal of Machine Learning Research. 2011. Vol. 12. P. 2825-2830.
4. Pace R. K., Barry R. Sparse Spatial Autoregressions // Statistics & Probability Letters. 1997. Vol. 33, No. 3. P. 291-297. DOI: 10.1016/S0167-7152(96)00140-X.
5. Rudin C. Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead // Nature Machine Intelligence. 2019. Vol. 1. P. 206-215. DOI: 10.1038/s42256-019-0048-x.
6. Ma X., Chen L., Deng Z. et al. Deep Image Feature Learning With Fuzzy Rules // IEEE Transactions on Emerging Topics in Computational Intelligence. 2024.

Vol. 8. P. 724-737. DOI: 10.1109/TETCI.2023.3259447.

7. Lebedeffson. deep-neuro-fuzzy [Электронный ресурс]. URL: <https://github.com/lebedeffson/deep-neuro-fuzzy> (дата обращения: 13.04.2026).